

Face Recognition

```
FRmodel = faceRecoModel(input_shape=(3, 96, 96))  
  
print("Total Params:", FRmodel.count_params())
```

Total Params: 3743280

臉部辨識的作業偏向體驗性質，因為 NN 已經寫好，只需 load 進來。

```
def verify(image_path, identity, database, model):  
    """  
    Function that verifies if the person on the "image_path" image is "identity".  
  
    Arguments:  
    image_path -- path to an image  
    identity -- string, name of the person you'd like to verify the identity. Has to be a resident of the  
    Happy house.  
    database -- python dictionary mapping names of allowed people's names (strings) to their encodings (ve  
    ctors).  
    model -- your Inception model instance in Keras  
  
    Returns:  
    dist -- distance between the image_path and the image of "identity" in the database.  
    door_open -- True, if the door should open. False otherwise.  
    """  
  
    ### START CODE HERE ###  
  
    # Step 1: Compute the encoding for the image. Use img_to_encoding() see example above. (~ 1 line)  
    encoding = img_to_encoding(image_path, model)  
  
    # Step 2: Compute distance with identity's image (~ 1 line)  
    dist = np.linalg.norm(encoding - database[identity])  
  
    # Step 3: Open the door if dist < 0.7, else don't open (~ 3 lines)  
    if dist < 0.7:  
        print("It's " + str(identity) + ", welcome home!")  
        door_open = True  
    else:  
        print("It's not " + str(identity) + ", please go away")  
        door_open = False  
  
    ### END CODE HERE ###  
  
    return dist, door_open
```

```
verify("images/camera_0.jpg", "younes", database, FRmodel)  
verify("images/camera_2.jpg", "kian", database, FRmodel)
```

It's younes, welcome home!
It's not kian, please go away

第一部分是 1 對 1 的比對，使用者拿 ID 卡讓機器知道自己是誰，機器再擷取臉部確認與資料庫的圖片是否相近，threshold 可以自己調整是否要嚴格，愈嚴格 dist 愈小。

```
def who_is_it(image_path, database, model):
    """
    Implements face recognition for the happy house by finding who is the person on the image_path image.

    Arguments:
    image_path -- path to an image
    database -- database containing image encodings along with the name of the person on the image
    model -- your Inception model instance in Keras

    Returns:
    min_dist -- the minimum distance between image_path encoding and the encodings from the database
    identity -- string, the name prediction for the person on image_path
    """

    ### START CODE HERE ###

    ## Step 1: Compute the target "encoding" for the image. Use img_to_encoding() see example above. ## (~
1 line)
    encoding = img_to_encoding(image_path,model)

    ## Step 2: Find the closest encoding ##

    # Initialize "min_dist" to a large value, say 100 (~1 line)
    min_dist = 100

    # Loop over the database dictionary's names and encodings.
    for (name, db_enc) in database.items():

        # Compute L2 distance between the target "encoding" and the current "emb" from the database. (~ 1
line)
        dist = np.linalg.norm(encoding-db_enc)

        # If this distance is less than the min_dist, then set min_dist to dist, and identity to name. (~
3 lines)
        if dist<min_dist:
            min_dist = dist
            identity = name

    ### END CODE HERE ###

    if min_dist > 0.7:
        print("Not in the database.")
    else:
        print ("it's " + str(identity) + ", the distance is " + str(min_dist))

    return min_dist, identity
```

```
who_is_it("images/camera_1.jpg", database, FRmodel)
```

```
it's bertrand, the distance is 0.46740144
(0.46740144, 'bertrand')
```

第二部分是 1 對多的辨識，不只要知道人臉再資料庫，也要判斷人臉是屬於誰，因此會需要計算到 triplet loss。

```
def triplet_loss(y_true, y_pred, alpha = 0.2):
    """
    Implementation of the triplet Loss as defined by formula (3)

    Arguments:
    y_true -- true labels, required when you define a loss in Keras, you don't need it in this function.
    y_pred -- python list containing three objects:
        anchor -- the encodings for the anchor images, of shape (None, 128)
        positive -- the encodings for the positive images, of shape (None, 128)
        negative -- the encodings for the negative images, of shape (None, 128)

    Returns:
    Loss -- real number, value of the loss
    """

    anchor, positive, negative = y_pred[0], y_pred[1], y_pred[2]

    ### START CODE HERE ### (~ 4 lines)
    # Step 1: Compute the (encoding) distance between the anchor and the positive, you will need to sum ov
er axis=-1
    pos_dist = tf.reduce_sum(tf.square(tf.subtract(anchor,positive)),axis=-1)
    # Step 2: Compute the (encoding) distance between the anchor and the negative, you will need to sum ov
er axis=-1
    neg_dist = tf.reduce_sum(tf.square(tf.subtract(anchor,negative)),axis=-1)
    # Step 3: subtract the two previous distances and add alpha.
    basic_loss = tf.add(tf.subtract(pos_dist,neg_dist),alpha)
    # Step 4: Take the maximum of basic_loss and 0.0. Sum over the training examples.
    loss = tf.reduce_sum(tf.maximum(basic_loss,0))
    ### END CODE HERE ###

    return loss
```

Neural Style Transfer

```
def compute_content_cost(a_C, a_G):
    """
    Computes the content cost

    Arguments:
    a_C -- tensor of dimension (1, n_H, n_W, n_C), hidden layer activations representing content of the image C
    a_G -- tensor of dimension (1, n_H, n_W, n_C), hidden layer activations representing content of the image G

    Returns:
    J_content -- scalar that you compute using equation 1 above.
    """

    ### START CODE HERE ###
    # Retrieve dimensions from a_G (≈1 line)
    m, n_H, n_W, n_C = a_G.get_shape().as_list() # m is the # of img

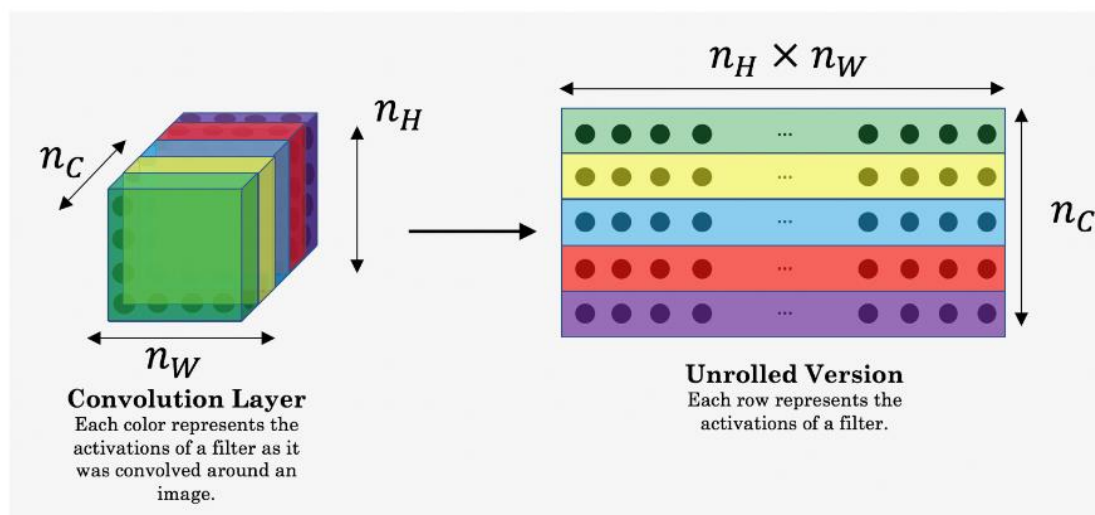
    # Reshape a_C and a_G (≈2 lines)
    a_C_unrolled = tf.transpose(tf.reshape(a_C, (n_H*n_W, n_C)))
    a_G_unrolled = tf.transpose(tf.reshape(a_G, (n_H*n_W, n_C)))

    # compute the cost with tensorflow (≈1 line)

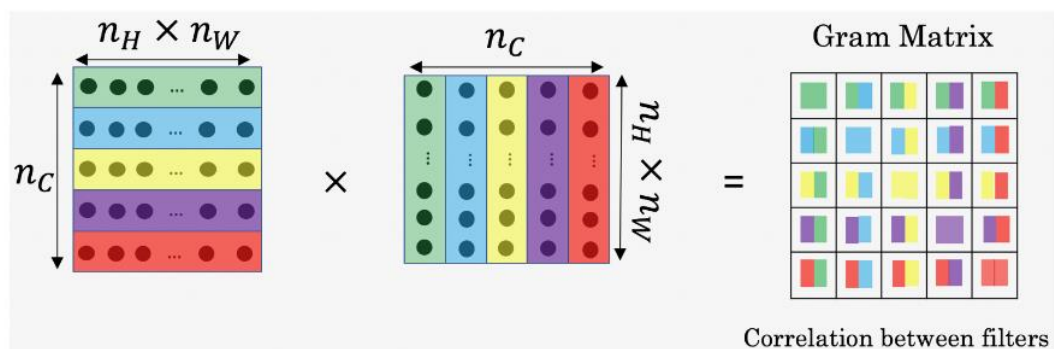
    J_content = tf.reduce_sum(tf.square(tf.subtract(a_C_unrolled, a_G_unrolled)))/(4*n_H*n_W*n_C)
    ### END CODE HERE ###

    return J_content
```

$J_{\text{content}} = 6.7655935$



輸入欲加上慮鏡的圖層，每一個 channel 攤平成 $H \times W$ ，共有 C 個 row，所以放入的過程中要先設定矩陣維度 $= (H \times W, C)$ ，而我們需要的是 $(C, H \times W)$ ，用 `transpose` 轉換，最後計算欲加上慮鏡的圖與生成的圖之間的 loss。



```
def gram_matrix(A):
    """
    Argument:
    A -- matrix of shape (n_C, n_H*n_W)

    Returns:
    GA -- Gram matrix of A, of shape (n_C, n_C)
    """

    ### START CODE HERE ### (~1 line)
    GA = tf.linalg.matmul(A,tf.transpose(A))
    ### END CODE HERE ###

    return GA
```

GA = [[6.422305 -4.429122 -2.096682]
[-4.429122 19.465837 19.563871]
[-2.096682 19.563871 20.686462]]

```
def compute_layer_style_cost(a_S, a_G):
    """
    Arguments:
    a_S -- tensor of dimension (1, n_H, n_W, n_C), hidden layer activations representing style of the image S
    a_G -- tensor of dimension (1, n_H, n_W, n_C), hidden layer activations representing style of the image G

    Returns:
    J_style_layer -- tensor representing a scalar value, style cost defined above by equation (2)
    """

    ### START CODE HERE ###
    # Retrieve dimensions from a_G (~1 line)
    m, n_H, n_W, n_C = a_G.get_shape().as_list()
    #sess=tf.Session()
    #print(sess.run(a_G))
    # Reshape the images to have them of shape (n_C, n_H*n_W) (~2 Lines)
    a_S = tf.transpose(tf.reshape(a_S, (n_H*n_W, n_C))) #不能用tf.reshape(a_S, (n_C, n_H*n_W)),因為n_H*n_W是column
    a_G = tf.transpose(tf.reshape(a_G, (n_H*n_W, n_C)))

    # Computing gram_matrices for both images S and G (~2 Lines)
    GS = tf.linalg.matmul(a_S,tf.transpose(a_S))
    GG = tf.linalg.matmul(a_G,tf.transpose(a_G))

    # Computing the Loss (~1 line)
    J_style_layer = tf.reduce_sum(tf.square(tf.subtract(GS,GG)))/(4*(n_H*n_W*n_C)*(n_H*n_W*n_C))

    ### END CODE HERE ###

    return J_style_layer
```

J = 35.34667875478276

而我們的風格圖計算 loss_g 使用以下算式：

$$J_{\text{style}}^{[l]}(S, G) = \frac{1}{4 \times n_C^2 \times (n_H \times n_W)^2} \sum_{i=1}^{n_C} \sum_{j=1}^{n_C} (G_{ij}^{(S)} - G_{ij}^{(G)})^2$$

所以必須先算出風格圖與生成圖的 gram matrix，將自身的 H*W 與 C 混合，再計算兩者的 loss。

STYLE_LAYERS 決定慮鏡的強度，數值愈大，慮鏡效果愈深。

```
STYLE_LAYERS = [
    ('conv1_1', 0.2),
    ('conv2_1', 0.2),
    ('conv3_1', 0.2),
    ('conv4_1', 0.2),
    ('conv5_1', 0.2)]
```

```

1 content_image = scipy.misc.imread("images/cat.jpg")
2 content_image = reshape_and_normalize_image(content_image)
3 style_image = scipy.misc.imread("images/drop-of-water.jpg")
4 style_image = reshape_and_normalize_image(style_image)
5 print(content_image.shape)
6 print(style_image.shape)
7 generated_image = generate_noise_image(content_image)
8 imshow(generated_image[0])

```

由於傳入的圖片每張大小都不一定是 NN 能吃進去的 size，我們必須先 resize 成 NN 能接受的大小(reshape_and_normalize_image)。

```
def compute_style_cost(model, STYLE_LAYERS):
```

```
def total_cost(J_content, J_style, alpha = 10, beta = 40):
```

這兩個 function 用來計算加入 model 後衍生出多層的 loss。

觀察完成合成的照片：



0 次



60 次



160 次

第 0 次只有 noise 影響，當初使值計算原圖與生成圖的 loss，隨著次數增多，因為權重較小，所以至少每 60 次才会有比較大的不同。

再來觀察貓貓套用沙漠慮鏡：



0 次



60 次



180 次

由於沙漠顏色與貓貓原圖差異很大，所以效果容易輸出，愈多次沙漠的地形也影響得愈顯著。



60 次水滴



60 次石頭



180 次水滴



180 次石頭

套用水滴和石頭的慮鏡，因為兩者的紋路較深，容易顯現出來，做太多次反而蓋過原圖，成效不好。